

Code Walkthrough

pathway_enrichment_498v510.R — Line by Line

Generated by Biomni | 2026-07-11

What This Script Does

This R script performs Gene Ontology (GO) enrichment analysis on DESeq2 differential expression results. It takes a CSV of DEG results, separates genes into upregulated and downregulated lists, tests each list for enrichment of GO Biological Process terms, and saves the results as CSV tables and dot plot figures. The script is wrapped in a reusable function so you can run the same analysis on any DEG CSV by changing one argument.

Package Color Key

base R	Built-in R functions (no package needed)
BiocManager	Installs Bioconductor packages
clusterProfiler	Runs enrichment analysis (enrichGO, enrichKEGG)
org.At.tair.db	Arabidopsis gene annotation database
AnnotationDbi	Interface to query annotation databases
enrichplot	Visualization functions for enrichment results
ggplot2	General plotting system (dotplot, ggtitle, ggsave)

Section 1: Package Installation (Lines 1-16)

The script begins by ensuring all required packages are installed. It uses a conditional pattern: check if the package is already installed, and only install it if missing. This makes the script portable.

```
base    if (!requireNamespace("BiocManager", quietly = TRUE))
base    install.packages("BiocManager")
```

`requireNamespace()` checks whether a package is installed without loading it. `'quietly = TRUE'` suppresses messages. The `'!'` negates the result: 'if BiocManager is NOT installed, run the next line.'

`install.packages()` downloads and installs a package from CRAN. BiocManager is the installer for Bioconductor (the repository for genomics R packages). We need it first because `clusterProfiler`, `org.At.tair.db`, etc. live on Bioconductor.

```
base    pkgs <- c("clusterProfiler", "org.At.tair.db", "enrichplot", "ggplot2")
base    for (p in pkgs) {
base      if (!requireNamespace(p, quietly = TRUE)) {
BiocManager BiocManager::install(p, update = FALSE, ask = FALSE)
base      }
base    }
```

Creates a character vector of the four required packages. `c()` is base R's combine function. `clusterProfiler` runs enrichment tests, `org.At.tair.db` provides Arabidopsis gene-to-GO mappings, `enrichplot` provides visualization, `ggplot2` is the plotting system.

Starts a for loop iterating over each package name. 'p' takes each value in turn.

Same check: if package 'p' is not installed, proceed to install it.

`BiocManager::install()` downloads from Bioconductor. The `'::'` syntax calls `install` from the `BiocManager` package specifically. `update = FALSE` prevents updating all other packages (slow). `ask = FALSE` prevents interactive prompts.

Closes the if.

Closes the for loop.

```
base    library(clusterProfiler)
base    library(org.At.tair.db)
base    library(enrichplot)
base    library(ggplot2)
```

`library()` loads a package so its functions are available without `'::'` prefix. `clusterProfiler` provides `enrichGO()`, `enrichKEGG()`, and other enrichment functions.

Loads the Arabidopsis thaliana annotation database. Maps TAIR locus IDs (AT1G01010) to GO terms, KEGG pathways, etc. Equivalent packages exist for human (`org.Hs.eg.db`), mouse (`org.Mm.eg.db`), etc.

Loads the enrichment visualization package. Provides `dotplot()`, `cnetplot()`, `emapplot()` for visualizing `clusterProfiler` results.

Loads the `ggplot2` plotting system. `enrichplot` builds on `ggplot2`, which also provides `ggtitle()` and `ggsave()` used directly in this script.

Section 2: Function Definition (Lines 18-26)

The script defines a reusable function so you can run the same analysis on different DEG files by changing arguments instead of editing code.

```
base run_pathway_enrichment <- function(deg_csv_path =  
base "C:/Users/.../DEG_...csv",  
base label,  
base padj_cutoff = 0.05,  
base lfc_cutoff = 0) {
```

Defines a function named `run_pathway_enrichment` using `function()`. The `<-` assigns it. The first argument, `deg_csv_path`, is the file path to a DESeq2 results CSV. It has a default value so the function can be called without it.

Second argument: a short label (e.g., 'OSD498_510_radiation') used in output filenames. No default — you must provide it.

Adjusted p-value threshold for significance. Default 0.05. Can be set to 0.01 for stricter results.

Optional log2 fold change threshold. Default 0 = no fold change filter. Setting to 1 requires at least 2-fold change.

Why use a function instead of a script?

Wrapping the analysis in a function means you can run it on all 6 DEG CSVs by calling the function 6 times with different arguments, instead of copying the entire script 6 times. If you find a bug, you fix it once.

Section 3: Loading & Validating the DEG Table (Lines 28-38)

The function loads the CSV and checks it has the expected columns. This validation prevents confusing errors later.

```
base cat("\n=====/\n")
base cat("Running enrichment for:", label, "\n")
base cat("Input file:", deg_csv_path, "\n")
base cat("=====\n")
```

cat() prints text to the console. \n creates a newline. This prints a visual separator so you can see where each run starts.

Prints the label so you know which comparison is being processed.

Prints the file path being analyzed.

Closing separator bar.

```
base res_df <- read.csv(deg_csv_path, row.names = 1)
base required_cols <- c("log2FoldChange", "padj")
base missing_cols <- setdiff(required_cols, colnames(res_df))
base if (length(missing_cols) > 0) {
base   stop("Input file is missing expected column(s): ", paste(missing_cols,
base     collapse = ", "),
base   " -- is this really a DESeq2 results CSV?")
base }
```

read.csv() reads a CSV into a data frame. row.names = 1 uses the first column (gene IDs like AT1G01010) as row names — which is how clusterProfiler expects them.

Creates a vector of column names the script expects: log2FoldChange for direction/magnitude, padj for significance.

setdiff() finds elements in the first argument NOT in the second. Here: which required columns are missing from the CSV. If all present, missing_cols is empty (length 0).

If any required columns are missing...

stop() halts execution with an error message. paste() joins missing column names with commas. Gives a clear error like: 'missing column(s): padj -- is this really a DESeq2 results CSV?'

Continuation of the error message.

Closes the if.

Section 4: Building the Universe & Gene Lists (Lines 40-52)

This section defines the 'universe' (all genes DESeq2 tested) and extracts significant gene lists. The universe is the background for enrichment testing — using all tested genes (not the whole genome) accounts for low-expression genes filtered out by DESeq2.

```
base universe <- rownames(res_df)
base cat("Universe size (genes tested):", length(universe), "\n")
```

Extracts all gene IDs from row names. This is the 'universe' or 'background' for the enrichment test. Should be all genes DESeq2 tested, not the whole genome, because filtered-out genes should not count as 'not enriched.'

Prints how many genes are in the universe (typically 20,000-25,000 for Arabidopsis).

```
base sig_up <- rownames(subset(res_df, padj < padj_cutoff & log2FoldChange >
base                               lfc_cutoff))
base sig_down <- rownames(subset(res_df, padj < padj_cutoff & log2FoldChange <
base                               -lfc_cutoff))
```

Creates the upregulated gene list. `subset()` filters rows matching a condition: `padj` below cutoff AND `log2FoldChange` above `lfc_cutoff` (positive = upregulated). `rownames()` extracts the gene IDs.

Creates the downregulated gene list. Same logic but `log2FoldChange < -lfc_cutoff` (negative = downregulated).

```
base cat("Upregulated genes (padj <", padj_cutoff,
base if (lfc_cutoff > 0) paste0(", |log2FC| > ", lfc_cutoff) else "", "):",
base length(sig_up), "\n")
base cat("Downregulated genes:", length(sig_down), "\n")
```

Prints the upregulated gene count with the threshold used.

Inline conditional: if `lfc_cutoff > 0`, appends the fold change threshold to the message; otherwise adds nothing.

Prints the count.

Prints the downregulated count.

```
base if (length(sig_up) == 0 && length(sig_down) == 0) {
base   cat("No significant genes found -- skipping enrichment.\n")
base   return(invisible(list(label = label, sig_up = sig_up, sig_down = sig_down,
base   ego_up = NULL, ego_down = NULL)))
base }
```

Checks if both gene lists are empty. '&&' means AND — both must be true. Handles cases where no genes meet the threshold (common with strict cutoffs or comparisons like GCR40).

Prints message and returns early.

`return()` exits early. `invisible()` prevents printing. Returns a list with the label and empty gene lists so the caller can inspect the result.

Enrichment objects set to NULL.

Closes the if.

Section 5: ID Mapping Sanity Check (Lines 60-72)

Before running enrichment, the script checks that gene IDs can be found in the annotation database. If IDs are in the wrong format, enrichment would silently return zero results. This catches that early.

```

base      check_ids <- head(c(sig_up, sig_down), 20)
base
base      if (length(check_ids) > 0) {
AnnotationDbi      mapped_check <- suppressWarnings(
org.At.tair.db      AnnotationDbi::select(org.At.tair.db, keys = check_ids,
                        columns = "GO", keytype = "TAIR")
base      )
base      n_mapped <- length(unique(mapped_check$TAIR))
base      cat("ID mapping sanity check:", n_mapped, "of", length(check_ids),
base      "sampled genes mapped to at least one GO term\n")
base      if (n_mapped == 0) {
base      warning("ZERO genes mapped. Check that your gene IDs are in TAIR
base      "locus format (e.g. AT1G01010) -- results will likely be empty.")
base      }
base      }

```

Takes the first 20 gene IDs from combined up + down lists. `head()` returns first N elements. `c()` combines the vectors. A sample is enough to verify ID format.

Proceeds only if there are genes to check.

`suppressWarnings()` runs the expression but hides warnings. `select()` sometimes warns about 1:many mappings that are not problems.

`AnnotationDbi::select()` queries an annotation database. `::` calls `select` from `AnnotationDbi` specifically (both `clusterProfiler` and `AnnotationDbi` have `select`). `'keys'` is the gene IDs to look up. `org.At.tair.db` is the database.

`'columns = "GO"'` retrieves GO terms. `'keytype = "TAIR"'` tells the database the IDs are TAIR locus format (AT1G01010). Change this if using a different ID format.

Closes `suppressWarnings()`.

Counts how many of the 20 sampled IDs were found. `unique()` removes duplicates (one gene maps to multiple GO terms). If 0, the ID format is wrong.

Prints the result.

Continuation.

If zero genes mapped, ID format is wrong.

`warning()` prints a warning but does NOT stop execution (unlike `stop()`). Alerts the user that enrichment will likely be empty.

End of warning.

Closes inner if.

Closes outer if.

Section 6: Running GO Enrichment (Lines 74-84)

This is the core — running the enrichment test. `enrichGO` performs a hypergeometric test (equivalent to Fisher's exact) to determine whether each GO term is over-represented in the significant gene list compared to the universe.

base	<code>ego_up <- if (length(sig_up) > 0) {</code>
clusterProfiler	<code> enrichGO(gene = sig_up, universe = universe, OrgDb = org.At.tair.db,</code>
clusterProfiler	<code> keyType = "TAIR", ont = "BP", pAdjustMethod = "BH",</code>
clusterProfiler	<code> pvalueCutoff = 0.05, qvalueCutoff = 0.2)</code>
base	<code>} else NULL</code>

Inline if-else: only run enrichment if there are upregulated genes. If empty, `ego_up = NULL`. Avoids errors from empty gene lists.

`enrichGO()` performs over-representation analysis for GO terms. 'gene' = significant genes to test. 'universe' = background. 'OrgDb' = annotation database (`org.At.tair.db` for Arabidopsis).

`keyType = "TAIR"`: gene IDs are TAIR locus format. `ont = "BP"`: Biological Process ontology (one of three GO domains; others are Molecular Function and Cellular Component). `pAdjustMethod = "BH"`: Benjamini-Hochberg FDR correction — standard for enrichment.

`pvalueCutoff = 0.05`: only terms with adjusted $p < 0.05$ retained. `qvalueCutoff = 0.2`: additional FDR filter. These determine how many enriched terms appear in output.

If no upregulated genes, set to `NULL`.

base	<code>ego_down <- if (length(sig_down) > 0) {</code>
clusterProfiler	<code> enrichGO(gene = sig_down, universe = universe, OrgDb = org.At.tair.db,</code>
clusterProfiler	<code> keyType = "TAIR", ont = "BP", pAdjustMethod = "BH",</code>
clusterProfiler	<code> pvalueCutoff = 0.05, qvalueCutoff = 0.2)</code>
base	<code>} else NULL</code>

Same pattern for downregulated genes.

Same `enrichGO` call with `sig_down`. Tests whether downregulated genes are enriched for specific biological processes.

Same parameters: TAIR IDs, BP ontology, BH correction.

Same cutoffs.

If no downregulated genes, set to `NULL`.

What `enrichGO` actually does

For each GO term (e.g., 'DNA repair'), `enrichGO` builds a 2x2 contingency table: (your genes in this term) vs (your genes not in this term) vs (universe genes in this term) vs (universe genes not in this term). It performs a hypergeometric test asking: 'Is the proportion of my gene list in this GO term significantly higher than expected by chance?' BH correction adjusts for testing hundreds of GO terms at once.

Section 7: Saving CSV & Figure Outputs (Lines 86-108)

After enrichment, the script saves results as CSV tables and dot plot figures. It checks whether any significant terms were found before saving.

```
base    if (!is.null(ego_up) && nrow(as.data.frame(ego_up)) > 0) {
base    out_csv <- paste0("GO_enrichment_", label, "_upregulated.csv")
base    write.csv(as.data.frame(ego_up), out_csv, row.names = FALSE)
base    cat("Saved:", out_csv, "(", nrow(as.data.frame(ego_up)), "enriched terms\n")
```

Checks: (1) ego_up is not NULL (enrichment was run), AND (2) at least one enriched term was found. as.data.frame() converts the enrichResult object to a data frame for nrow().

Constructs the output filename. paste0() joins strings with no separator. For label='OSD498_510_radiation', produces 'GO_enrichment_OSD498_510_radiation_upregulated.csv'.

write.csv() writes the data frame to CSV. row.names = FALSE prevents an extra column of row numbers.

Prints confirmation with filename and term count.

```
enrichplot  p_up <- dotplot(ego_up, showCategory = 15) +
ggplot2      ggtitle(paste0(label, " -- UPregulated"))
ggplot2      ggsave(paste0("GO_dotplot_", label, "_upregulated.png"), p_up, width = 9,
base          height = 7)
base          } else {
base          cat("No enriched terms found for upregulated genes (or no genes to
base          test).\n")
base          }
```

dotplot() from enrichplot creates a dot plot: each dot = a GO term, x-axis = GeneRatio, dot size = gene count, dot color = p-value. showCategory = 15 shows top 15 terms. The '+' adds a ggplot2 layer.

ggtitle() from ggplot2 adds a title. paste0() combines the label with ' -- UPregulated'.

ggsave() from ggplot2 saves the plot as PNG (detected from .png extension). Width and height in inches.

If no enriched terms found...

Prints message instead of creating empty files.

Closes the if-else.

The same pattern repeats for downregulated genes (lines 98-108): check if ego_down has results, save CSV, create dotplot with 'DOWNregulated' title, save figure. If no terms found, print a message.

```
base    invisible(list(label = label, sig_up = sig_up, sig_down = sig_down,
base    ego_up = ego_up, ego_down = ego_down))
```

invisible() returns a value without printing it. The function returns a list with the label, gene lists, and enrichment objects so you can inspect them later (e.g., results_5a\$ego_up).

Enrichment objects included for later access.

Section 8: Calling the Function (Lines 110-130)

The script ends by calling the function on each comparison. Most calls are commented out — uncomment the ones you want to run.

```
base      setwd("C:/Users/smith/.../pathway enrichment csv/658_GCR80")
```

setwd() sets the working directory. All output files (CSVs, PNGs) save here. Change this path for each comparison's output location.

```
base      results_5d_80 <- run_pathway_enrichment(  
base      deg_csv_path = "C:/Users/smith/.../DEG_OSD658_GCR80_vs_none.csv",  
base      label = "OSD658_GCR80")
```

Calls the function and stores the result in results_5d_80. The '<-' captures the returned list for later inspection.

Path to the DESeq2 results CSV for this comparison.

Label for output filenames. Produces files like 'GO_enrichment_OSD658_GCR80_upregulated.csv'.

Section 9: How to Read the Output Table (Lines 132-142)

The script ends with a comment block explaining the output CSV columns. Understanding these is essential for interpreting results.

Column	Meaning
Description	The biological process name (e.g., 'DNA repair', 'response to radiation')
GeneRatio	(your genes in this term) / (total genes in your list). Higher = more of your genes in this term.
BgRatio	(genes in this term overall) / (total genes in universe). The background rate.
pvalue	Raw p-value from the hypergeometric test. Lower = more significant.
p.adjust	Adjusted p-value after Benjamini-Hochberg correction. Filter on this (< 0.05).
qvalue	Another FDR estimate. The script filters at q < 0.2.
geneID	Specific gene IDs from your list in this GO term (slash-separated)
Count	How many of your genes are in this term (numerator of GeneRatio)

How to judge real enrichment

A GO term is meaningfully enriched when GeneRatio is notably higher than BgRatio AND $p.adjust < 0.05$. Example: GeneRatio = 15/100 (15% of your genes) vs BgRatio = 50/20000 (0.25% of universe) = 60x enrichment. But if GeneRatio = 50/100 and BgRatio = 10000/20000 (both 50%), there's no enrichment even with many genes.

Key Parameters You Can Change

Parameter	Current	Alternative	Effect of changing
padj_cutoff	0.05	0.01	Stricter: fewer genes, more conservative enrichment
lfc_cutoff	0	1.0	Only enrich genes with ≥ 2 -fold change
ont	"BP"	"MF" or "CC"	Molecular Function or Cellular Component ontology
pAdjustMethod	"BH"	"bonferroni"	Stricter correction (fewer enriched terms)
showCategory	15	30	Show more terms in the dot plot